BANGLADESH UNIVERSITY OF BUSINESS & TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

3D Car Game

INTAKE-50
SECTION-04

Prepared By:

Samia Rahman
22234103338

Asha Haider Heeya
22234103340

Lamia Binte Momin
22234103347

Maksuda Parvin
22234103375

Md. Fahad bin Hossain Foysal
22234103375

Supervisor:
Fahmida Ahmed
Lecturer, CSE, BUBT

1. Introduction

This project presents a fully interactive 3D Car Driving Simulator designed using C++, OpenGL, and GLUT. The simulator combines graphics rendering, physics simulation, and game design mechanics to create an immersive environment where the player drives a car, collects coins, avoids obstacles, and experiences dynamic seasonal & weather changes.

The project demonstrates advanced Computer Graphics concepts such as:

- 3D modeling
- Lighting & shading
- Fog & weather simulation
- Camera transformations
- Particle systems
- Matrix manipulations
- Event-driven programming via GLUT

This project successfully integrates all major OpenGL pipelines to produce a realistic and engaging game.

2. Objectives

The main objectives of the simulator are:

1. Create a 3D environment that simulates different seasons, weather, and day/night cycles.
2. Implement realistic car physics (acceleration, braking, steering, friction).
3. Develop collision detection, coin collection, and obstacle avoidance.
4. Design multiple camera modes for an enhanced gameplay experience.
5. Apply lighting, shading, blending, fog, and particle effects using OpenGL.
6. Ensure smooth animation using GLUT timers & double buffering.

3. System Requirements

Software

- C++
- OpenGL
- GLUT/FreeGLUT
- CodeBlocks / Visual Studio / Dev-C++

4. Project Features

Gameplay

- Collect 50 coins
- Avoid obstacles
- Realistic physics engine
- Speed, score & timer tracking

Environment

- 6 Seasons: Summer, Rainy, Autumn, Late Autumn, Winter, Spring
- Weather: Rain, Snow, Falling Leaves
- Day/Night cycle (every 15 seconds)

Camera Modes

- Follow camera
- First-person
- Top-down
- Free camera

Visual Effects

- Lighting
- Fog
- Clouds, sun, moon, stars
- Snow & rain particle systems

Car Model

- Detailed 3D car
- Rotating wheels, steering animation
- Working headlights & taillights
- Color customization

HUD & UI

- Score
- Speed
- Season name
- Time
- Camera mode
- Instructions panel

5. Explanation of All GLUT & OpenGL Functions Used

5.1 GLUT Initialization Functions

- **glutInit():** Initializes the GLUT library and processes command-line arguments.
- **glutInitDisplayMode():** Sets up display mode (Double buffering, RGB mode, Depth buffer).
- **glutInitWindowSize() / glutInitWindowPosition():** Defines initial window size and screen position.
- **glutCreateWindow():** Creates the main application window.

5.2 Callback Registration Functions

- **glutDisplayFunc(display):** Registers the main display/rendering function.
- **glutReshapeFunc(reshape):** Handles window resizing.
- **glutKeyboardFunc() / glutKeyboardUpFunc():** Detects normal key press & release.
- **glutSpecialFunc() / glutSpecialUpFunc():** Handles special keys (arrows, function keys).
- **glutTimerFunc():** Used for animation; updates game logic every 16ms (~60 FPS).

5.3 Geometry Drawing Functions

- **glutSolidCube():** Car body, obstacles
- **glutSolidSphere():** Sun, moon, wheels, rocks
- **glutSolidCone():** Traffic cones
- **glutSolidTorus():** Tire structure
- **glBegin(GL_QUADS/LINES/POINTS):** Road, borders, sky, particles

5.4 Transformation Functions

- **glTranslatef():** Move object
- **glRotatef():** Rotate object
- **glScalef():** Resize object
- **glPushMatrix()/glPopMatrix():** Save & restore matrix states

5.5 Viewing & Projection

- **gluPerspective():** Creates perspective projection (3D view).
- **gluLookAt():** Sets camera position, target, and up direction.
- **gluOrtho2D():** Used for HUD (2D overlay).

5.6 Lighting Functions

- **glEnable(GL_LIGHTING)**

- **glEnable(GL_LIGHT0)**
- **glLightfv():** Set ambient, diffuse, specular, and position
- **glColorMaterial():** Allows glColor to control lighting material

5.7 Fog Functions

- **glEnable(GL_FOG)**
- **glFogf(GL_FOG_DENSITY):** Fog thickness
- **glFogfv(GL_FOG_COLOR)**
- **glFogi(GL_FOG_MODE) (EXP2 for realism)**

5.8 Transparency / Blending

- **glEnable(GL_BLEND):** Used for rain, snow, smoke, and HUD background panels.
- **glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):** Enables semi-transparent effects.

5.9 Depth Buffering

- **glEnable(GL_DEPTH_TEST):** Needed for 3D rendering
- **glClear(GL_DEPTH_BUFFER_BIT):** Clears depth memory

5.10 Bitmap Text

- **glutBitmapCharacter():** Renders 2D text in HUD.

5.11 Main Loop

- **glutMainLoop():** Starts event-driven infinite loop.

6. Project Architecture

Main Components

1. Car physics engine
2. Camera controller
3. Environment renderer
4. Particle system (rain/snow/leaves)
5. Collision system
6. HUD system
7. User input handler
8. Seasonal & weather manager
9. Lighting and fog engine

7. Game Flowchart

1. Initialize GLUT + OpenGL
2. Load assets & variables
3. Start timer
4. Render environment
5. Render car
6. Handle inputs
7. Detect collision/collection
8. Update camera
9. Display HUD
10. Swap buffers
11. Repeat

8. Team Contributions

Member Name	Responsibility	Detailed Tasks Performed
Lamia Binte Momin	Environment	● Initialize environment ● Skybox, sun, moon, stars render ● Trees, grass & ground color shading
Maksuda Parvin	Car Physics & Collision System	● Full car physics engine (acceleration, braking, friction) ● Steering + wheel rotation mechanics ● Collision detection with obstacles & coins ● Boundary detection & road looping system
Samia Rahman	Camera, Lighting & Season System	● 4 camera modes implementation ● Lighting system (ambient, diffuse, specular) ● 6 seasonal systems implement ● Seasonal weather + color transitions
Asha Haider Heeya	3D Models & Particle Effects	● Car body, wheels, and headlights modeling ● Tree, rock, traffic cone, barrier models ● Rain, snow, falling-leaf particle systems ● Seasonal particle density management
Fahad Bin Hossain	HUD, Game Logic &	● Score, speedometer, timer UI ● Instructions pop-up panel

	Controls	● Game state logic (win, lose, restart) ● Keyboard control system & camera switching

9. Challenges Faced

- Maintaining smooth FPS with many 3D objects
- Implementing realistic physics without external libraries
- Managing multiple camera modes
- Complex lighting behaviors during day/night
- Synchronizing particle effects with seasons

10. Conclusion

This project successfully demonstrates the use of advanced Computer Graphics techniques through a fully functional 3D driving simulator.
It integrates physics, 3D modeling, animation, lighting, and user interaction into a cohesive and visually rich application.

The project helped the team understand:

- OpenGL rendering pipeline
- Event-driven programming in GLUT
- Real-time graphics optimization
- 3D transformations and projections
- Simulation of natural phenomena